Facilitator answer key

Match the Git command to the GUI action

Do not hand this sheet to learners. Print or open the learner sheet (`command-gui-matching.qmd`) for them.

Answer key

Command	Answer	GUI action
1. `git clone <url>`	**D**	File > New Project > Version Control > Git
2. `git add` / stage	**F**	Tick the checkbox next to a file in the Git pane
3. `git commit -m "..."`	**B**	Write a message, click Commit
4. `git push`	**E**	Up-arrow (push) button in the Git pane
5. `git pull`	**A**	Down-arrow (pull) button in the Git pane
6. `git status`	**H**	Read the file list and status flags in the Git pane
7. `git log`	**K**	History (clock icon) in the Git pane
8. `git branch` / checkout	**I**	Branch dropdown > New Branch
9. `git revert <commit>`	**J**	Right-click a commit in History > Revert
10. `git add` + `commit` on GitHub	**C**	Add file / Upload files, then Commit changes
11. open a pull request *(GitHub feature)*	**G**	Pull requests tab > New pull request

Talking points while reviewing

- Every button press ran a real command. Nothing was hidden or “easier”; it was the same operation.
- `git add` and `git commit` are two steps. In the RStudio pane you see this clearly: tick the box (add), then Commit. On GitHub the web editor bundles both into one “Commit changes” click.
- `git revert` makes a new commit that undoes an earlier one; it does not erase history. In the GUI this is a right-click on a commit in History. Good moment to contrast it with deleting or force-pushing.
- A pull request is not a Git command in the same sense; it is a GitHub feature built on top of branches you pushed. That is why it is called out separately on the sheet. Good moment to name where Git ends and GitHub begins.

- Tie-in to the agentic follow-up: an agent driving Git speaks these same command names. Knowing the mapping is what lets you read and check what the agent does.